

Kubernetes

Formation Avancée — 3 jours

Programme complet des 3 jours

Jour	Modules	Approche
1	Architecture K8s, Pods, Services, Contrôleurs, Volumes, ConfigMaps/Secrets	40% théorie / 60% pratique
2	Authentification, Autorisation RBAC, Sécurité réseau/runtime, Déploiement avancé, ArgoCD, Helm	Démos guidées + exercices
3	Operators, CRDs, Admission Webhooks, API Aggregation, Scheduler Extensions, TP final	Conception + TP intégrateur

Cluster de lab : Kind multi-nœuds (1 control-plane + 3 workers)

Environnement de formation

```
./scripts/setup-lab.sh      # Docker, kubectl, Kind, Helm, ArgoCD CLI, k9s  
./scripts/create-cluster.sh # 1 CP + 3 workers, Ingress, Metrics Server  
kubectl get nodes -o wide
```

Outil	Rôle
kubectl	CLI pour interagir avec le cluster
Kind	Kubernetes in Docker (clusters locaux)
Helm	Gestionnaire de packages Kubernetes
k9s	Interface terminal interactive
ArgoCD CLI	Gestion GitOps

Module 1 — Architecture Kubernetes

Comprendre tous les composants du cluster

Kubernetes : origines et positionnement

- 2003-2014 : Google utilise **Borg** en interne (orchestration massive)
- Juin 2014 : Google open-source **Kubernetes** (inspiré de Borg)
- Juillet 2015 : K8s 1.0, transféré à la **CNCF**
- Écrit en **Go**, licence Apache 2.0

Ce que Kubernetes fait :

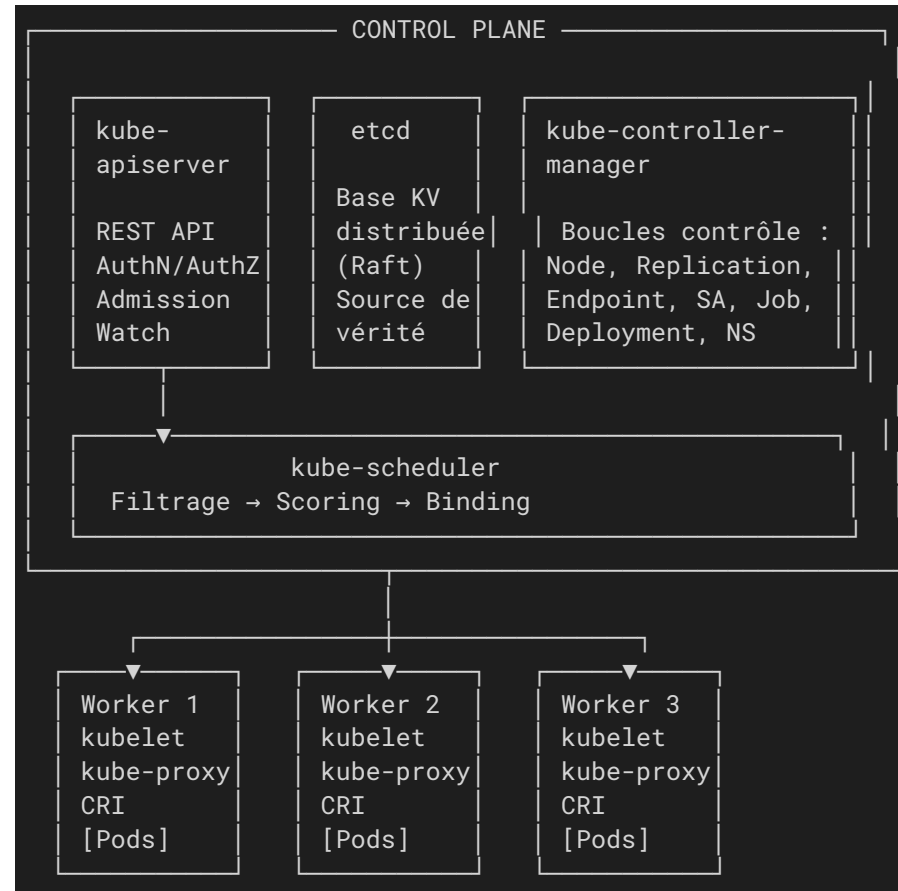
- Orchestration de conteneurs sur un cluster de machines
- Déploiement déclaratif (état souhaité en YAML)
- Self-healing (restart, remplacement, reschedule automatiques)
- Scaling horizontal et vertical
- Service discovery et load balancing intégrés
- Gestion des configurations et des secrets
- Rolling updates et rollbacks sans downtime
- Orchestration du stockage (PV/PVC)

Kubernetes : origines et positionnement

Ce que Kubernetes n'est PAS :

- Un PaaS (pas d'opinion sur les langages, frameworks)
- Un CI/CD (mais s'intègre avec Jenkins, GitLab CI, ArgoCD) et Un runtime de conteneurs (il utilise containerd ou CRI-O)

Architecture globale du cluster



kube-apiserver — Le point d'entrée unique

Rôle : serveur REST, unique point de communication. Tout passe par lui.

Pipeline de traitement d'une requête :

1. Réception HTTP (REST)
2. Authentication : Qui ? (Cert X.509, Token, OIDC)
3. Authorization : Droit ? (RBAC)
4. Mutating Admission : Modification (injection sidecar, defaults)
5. Schema Validation : YAML valide ?
6. Validating Admission : Politiques custom (sécurité)
7. Persistance dans etcd

Caractéristiques :

- Seul composant qui communique avec etcd
- Supporte le **watch** (notifications temps réel)
- Scalable horizontalement (multi-instances + LB)
- Extensible via Admission Webhooks (Jour 3)

kube-apiserver — Le point d'entrée unique

```
kubectl api-resources          # Toutes les ressources
kubectl api-resources --namespaced=false # Cluster-wide uniquement
kubectl explain pod.spec.containers # Documentation intégrée
kubectl proxy --port=8001 &      # Proxy REST local
curl -s http://localhost:8001/healthz
```

etcd — La base de données distribuée

etcd stocke TOUT l'état du cluster dans un format clé-valeur.

- Algorithme de consensus : **Raft** (leader election, log replication)
- Performance : ~10 000 écritures/seconde
- Production : 3 ou 5 instances (tolérance = $(n-1)/2$ pannes)

Structure des clés :

```
/registry/pods/default/mon-pod  
/registry/services/default/mon-service  
/registry/deployments/dev/mon-app  
/registry/secrets/production/db-creds  
/registry/configmaps/dev/app-config
```

etcd — La base de données distribuée

```
# Vérifier etcd dans Kind
kubectl get pods -n kube-system -l component=etcd
kubectl exec -n kube-system etcd-formation-k8s-control-plane -- \
  etcdctl endpoint health \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

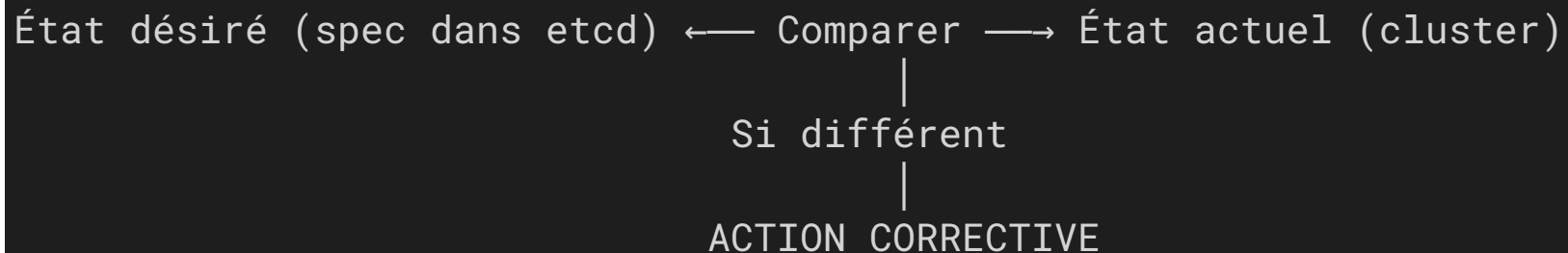
“ **Critique** : perte d'etcd = perte du cluster. Toujours faire des backups :

```
etcdctl snapshot save backup.db
```

”

kube-controller-manager — Les boucles de contrôle

Exécute les boucles de **réconciliation** en permanence :



kube-controller-manager – Les boucles de contrôle

Controller	Rôle détaillé
Node Controller	Détecte les nœuds NotReady, évicte les pods après timeout (5 min)
Replication Controller	Maintient le nombre de rélicas (crée ou supprime des pods)
Endpoints Controller	Met à jour les endpoints (IPs des pods) de chaque Service
ServiceAccount Controller	Crée le SA "default" dans chaque nouveau namespace
Namespace Controller	Nettoie les ressources quand un namespace est supprimé
Job Controller	Crée les pods pour les Jobs, vérifie les completions
Deployment Controller	Gère les ReplicaSets, orchestre les rolling updates
StatefulSet Controller	Gère les pods ordonnés avec identité stable
DaemonSet Controller	Assure qu'un pod tourne sur chaque nœud éligible
PV Binder Controller	Associe les PVC aux PV disponibles
CronJob Controller	Crée des Jobs selon le schedule cron

kube-scheduler — Algorithme de placement

Le Scheduler assigne les pods

Pending

(sans nœud) au meilleur nœud.

Phase 1 — Filtrage (nœuds éligibles) :

Plugin	Question posée
NodeResourcesFit	Assez de CPU et RAM ?
NodeName	Pod demande un nœud spécifique ?
NodeAffinity	Affinités requises respectées ?
TaintToleration	Pod tolère les taints ?
PodTopologySpread	Distribution topologique OK ?
VolumeBinding	Volume accessible depuis ce nœud ?

kube-scheduler — Algorithme de placement

Phase 2 — Scoring (classement 0-100) :

Plugin	Critère
NodeResourcesBalancedAllocation	Équilibrage CPU/RAM entre nœuds
ImageLocality	Image déjà en cache sur le nœud ?
InterPodAffinity	Préférences de co-localisation
NodeAffinity	Préférences de nœuds (preferred)

Phase 3 — Binding : le pod est assigné au nœud avec le meilleur score total.

Composants des Worker Nodes

kubelet — L'agent sur chaque nœud

- Watch l'API Server pour les nouveaux pods assignés à son nœud
- Télécharge les images via le CRI (containerd)
- Crée le sandbox réseau (pause container)
- Exécute les init containers séquentiellement
- Démarre les containers principaux en parallèle
- Exécute les probes (startup, liveness, readiness)
- Rapporte le statut à l'API Server
- Gère les volumes (montage/démontage)
- Gère la terminaison (SIGTERM → grace period → SIGKILL)

Composants des Worker Nodes

kube-proxy – Le gestionnaire réseau

Mode	Performance	Mécanisme
iptables	Bon (<1000 svc)	Règles DNAT, round-robin probabiliste
IPVS	Excellent	Hash table, algos : rr, lc, dh, sh, sed
nftables	Excellent (K8s 1.29+)	Successeur d'iptables

Container Runtime (CRI)

- **containerd** : défaut depuis K8s 1.24 (Docker retiré)
- **CRI-O** : utilisé par OpenShift

Le flux complet : `kubectl apply -f pod.yaml`

1. `kubectl` parse le YAML → POST `/api/v1/namespaces/default/pods`
2. API Server :
 - a) Authentication (cert, token, OIDC)
 - b) Authorization (RBAC : peut créer un pod ?)
 - c) Mutating Admission (injection sidecar, defaults)
 - d) Schema Validation (YAML conforme ?)
 - e) Validating Admission (policies custom)
3. Stockage dans etcd : `status.phase = "Pending"`, `nodeName = null`
4. Scheduler détecte le pod sans nœud (watch)
→ Filter → Score → Bind au meilleur nœud
5. kubelet du nœud sélectionné (watch) :
 - a) Pull image (si pas en cache)
 - b) Crée le sandbox réseau (pause container + net ns)
 - c) Exécute les init containers séquentiellement
 - d) Démarre les containers principaux en parallèle
 - e) Démarre les startup probes
 - f) Startup OK → liveness + readiness probes
 - g) Rapporte `status.phase = "Running"`

Positionnement DevOps / GitOps

```
Développeur → git push → CI Pipeline (build, test, lint, SAST)
|
Push image → Registry (Docker Hub, ECR, GCR)
|
Update manifest → Git repo infra
|
ArgoCD détecte le changement (watch Git)
|
Compare Git (désiré) vs Cluster (actuel)
|
Si différent → Sync automatique
|
Kubernetes applique l'état désiré
|
Monitoring → Alertes → Feedback
```

Positionnement DevOps / GitOps

Principes GitOps :

- Git = source de vérité unique (pas kubectl apply manuellement)
- Déclaratif (YAML, pas de scripts impératifs)
- Réconciliation automatique et continue (ArgoCD, Flux)
- Audit trail complet (historique Git = historique infra)

Principes fondamentaux de Kubernetes

Principe	Explication
Déclaratif	On décrit l'état souhaité, K8s converge
Réconciliation	Boucle infinie : observer → comparer → agir
Self-healing	Pod mort → recréé, nœud mort → pods reschedulés
Immutabilité	On ne modifie pas un conteneur, on en déploie un nouveau
Idempotence	kubectl apply peut être exécuté N fois sans effet de bord
Labels	Système de sélection universel (pas de hiérarchie rigide)
Loose coupling	Composants communiquent via l'API Server uniquement
Extensibilité	CRDs, Operators, Webhooks, Scheduler plugins

Module 2 — Les Pods

L'unité fondamentale de Kubernetes

Qu'est-ce qu'un Pod ?

Un **Pod** est la plus petite unité déployable. Il encapsule 1+ conteneurs qui partagent :

- **Réseau** : même IP, même port space, communication via `localhost`
- **Volumes** : les volumes du pod sont accessibles par tous les conteneurs
- **IPC** : espace de communication inter-processus partagé
- **PID namespace** (optionnel) : visibilité des processus entre conteneurs



Manifest Pod — Partie 1 : metadata et spec de base

```
apiVersion: v1                # "v1" = core API group
kind: Pod                     # Type de ressource
metadata:
  name: mon-pod               # Nom unique dans le namespace
  namespace: dev              # Namespace (défaut: "default")
  labels:                     # Labels pour sélection (Services, Deployments)
    app: frontend
    version: v2.1.0
    team: web
    tier: frontend
    app.kubernetes.io/name: frontend      # Labels recommandés K8s
    app.kubernetes.io/version: "2.1.0"
    app.kubernetes.io/component: web
    app.kubernetes.io/managed-by: kubectl
  annotations:                # Métadonnées non-sélectables (infos, config)
    description: "Frontend React SPA"
    git-commit: "abc123def456"
    prometheus.io/scrape: "true"
    prometheus.io/port: "9090"
spec:
  restartPolicy: Always        # Always | OnFailure | Never
  terminationGracePeriodSeconds: 30 # Temps pour arrêt propre
```


Manifest Pod – Partie 2 : containers

```
containers:
- name: app                # Nom unique dans le pod
  image: nginx:1.25        # registry/image:tag
  imagePullPolicy: IfNotPresent # Always | IfNotPresent | Never
  ports:
    - name: http           # Nom référençable par les Services
      containerPort: 80
      protocol: TCP
  command: ["/bin/sh"]      # Override ENTRYPOINT du Dockerfile
  args: ["-c", "nginx -g 'daemon off;'" ] # Override CMD
  workingDir: /app
  env:
    - name: APP_ENV
      value: "production"    # Valeur statique
    - name: POD_NAME
      valueFrom:
        fieldRef:
          fieldPath: metadata.name # Champ du pod (Downward API)
    - name: NODE_NAME
      valueFrom:
        fieldRef:
          fieldPath: spec.nodeName
    - name: POD_IP
      valueFrom:
        fieldRef:
          fieldPath: status.podIP
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-creds      # Depuis un Secret
          key: password
```

Manifest Pod – Partie 3 : resources

```
resources:
  requests:           # MINIMUM garanti (utilisé par le Scheduler)
    cpu: "100m"       # 100 millicores = 0.1 CPU
    memory: "128Mi"   # 128 Mébioctets
  limits:             # MAXIMUM autorisé
    cpu: "500m"       # 0.5 CPU – throttled si dépassé
    memory: "256Mi"   # OOMKilled si dépassé !
```

Unités CPU :

1

= 1 vCPU/core,

100m

= 0.1 CPU,

250m

= 0.25 CPU

Unités mémoire :

Ki

(kibi),

Mi

(mebi),

Gi

Manifest Pod – Partie 3 : ressources

QoS Classes :

Classe	Condition	Éviction
Guaranteed	requests == limits (CPU + RAM, tous containers)	Dernière
Burstable	requests < limits (au moins 1 container a des requests)	Intermédiaire
BestEffort	Aucun request ni limit défini	Première

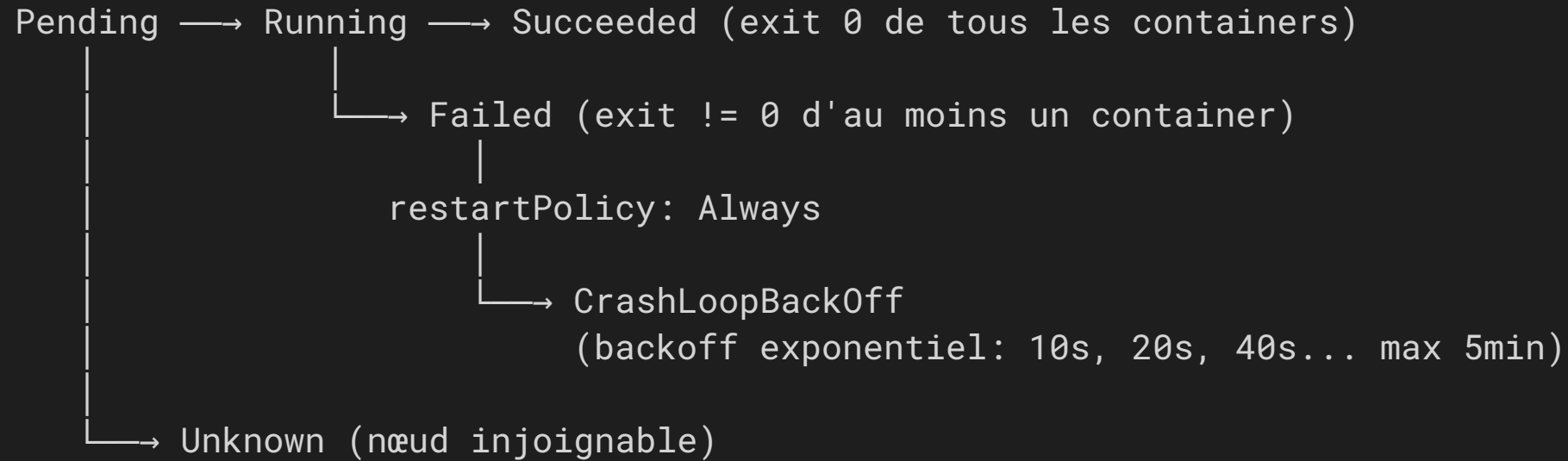
“ **Recommandation** : toujours définir des

requests

pour le scheduling. Les pods BestEffort sont évictés en premier en cas de pression mémoire.

”

Cycle de vie du Pod – Phases



Cycle de vie du Pod – Phases

Conditions du Pod :

Condition	Signification
PodScheduled	Pod assigné à un nœud
Initialized	Tous les init containers terminés (exit 0)
ContainersReady	Tous les conteneurs passent la readiness
Ready	Pod prêt à recevoir du trafic (dans les endpoints du Service)

```
kubectl get pod mon-pod -o jsonpath='{.status.phase}'
kubectl get pod mon-pod -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

Restart Policy et processus de terminaison

restartPolicy	Deployment	Job	Pod standalone
Always	Défaut	Interdit	Restart toujours
OnFailure	Possible	Défaut	Restart si exit != 0
Never	Possible	Possible	Jamais de restart

Restart Policy et processus de terminaison

Processus de terminaison détaillé :

1. `kubectl delete pod mon-pod`
2. API Server marque le pod "Terminating"
3. Pod RETIRÉ des endpoints de tous les Services (plus de trafic)
4. `preStop` hook exécuté (si défini)
5. `SIGTERM` envoyé au PID 1 de chaque conteneur
6. Attente de `terminationGracePeriodSeconds` (30s défaut)
7. Si toujours vivant → `SIGKILL` (arrêt forcé)
8. Pod supprimé d'`etcd`

```
lifecycle:  
  preStop:  
    exec:  
      command: ["/bin/sh", "-c", "nginx -s quit && sleep 5"]
```

Init Containers — Exécution séquentielle

Les **init containers** s'exécutent **un par un avant** les conteneurs principaux.

```
spec:
  initContainers:
    - name: wait-for-db
      image: busybox:1.36
      command: ['sh', '-c',
        'until nc -z postgresql.default.svc.cluster.local 5432;
        do echo "Waiting for DB..."; sleep 2; done']
    - name: run-migrations
      image: flyway/flyway:10
      args: ["migrate"]
      env:
        - name: FLYWAY_URL
          value: "jdbc:postgresql://postgresql:5432/mydb"
    - name: download-config
      image: curlimages/curl
      command: ['sh', '-c',
        'curl -o /config/app.yml https://config-server/api/config']
      volumeMounts:
        - name: config
          mountPath: /config
  containers:
    - name: app
      image: mon-app:latest
      volumeMounts:
        - name: config
```


Init Containers – Exécution séquentielle

Aspect	Init Containers	Containers principaux
Exécution	Séquentielle	Parallèle
Durée	Se terminent (exit 0)	Tournent en continu
Probes	Non supportées	liveness, readiness, startup

Pod multi-conteneurs – Les 3 patterns

Sidecar (le plus courant)

Enrichit le conteneur principal sans le modifier :

- **Envoy/Istio** : proxy réseau (mTLS, tracing, retry)
- **Fluentd/Fluent Bit** : collecte de logs
- **Vault Agent** : injection de secrets
- **Prometheus exporter** : métriques custom

Ambassador (proxy sortant)

L'app parle à

localhost

, l'ambassador route vers la bonne destination :

- **Cloud SQL Proxy** (Google) : proxy vers Cloud SQL
- **Proxy multi-env** : route vers dev/staging/prod selon la config

Pod multi-conteneurs – Les 3 patterns

Adapter (transformateur)

Transforme les sorties du conteneur principal :

- **Prometheus adapter** : convertit des métriques custom → format Prometheus
- **Log reformatter** : reformate les logs dans un format standard

Probes — Les 3 types détaillés

```
containers:
- name: app
  # STARTUP PROBE : vérifie que l'app a démarré
  # Tant que cette probe n'a pas réussi, les autres ne démarrent PAS
  startupProbe:
    httpGet: {path: /healthz, port: 8080}
    failureThreshold: 30    # 30 x 10s = 5 min pour démarrer
    periodSeconds: 10

  # LIVENESS PROBE : l'app est-elle vivante ?
  # Échec → RESTART du conteneur
  livenessProbe:
    httpGet: {path: /healthz, port: 8080}
    initialDelaySeconds: 0
    periodSeconds: 10
    timeoutSeconds: 3
    failureThreshold: 3    # 3 échecs → restart
    successThreshold: 1

  # READINESS PROBE : peut-elle recevoir du trafic ?
  # Échec → RETIRÉ des endpoints du Service (pas de trafic)
  readinessProbe:
    httpGet: {path: /ready, port: 8080}
```

Probes – Types de vérification

Type	Syntaxe	Succès	Cas d'usage
httpGet	<code>httpGet: {path: /healthz, port: 8080}</code>	HTTP 200-399	APIs REST, apps web
tcpSocket	<code>tcpSocket: {port: 5432}</code>	Port ouvert	BDD (PostgreSQL, Redis)
exec	<code>exec: {command: ["pg_isready"]}</code>	Exit code 0	Commandes custom
grpc	<code>grpc: {port: 50051}</code>	SERVING (K8s 1.24+)	Services gRPC

Probes – Types de vérification

```
# Redis – tcpSocket
livenessProbe:
  tcpSocket: {port: 6379}
  periodSeconds: 10

# PostgreSQL – exec
readinessProbe:
  exec:
    command: ["pg_isready", "-U", "postgres"]
  periodSeconds: 5

# gRPC – native
readinessProbe:
  grpc: {port: 50051, service: "my.health.v1.Health"}
  periodSeconds: 10
```

Labels et sélecteurs – Syntaxe complète

ÉGALITÉ

```
kubectl get pods -l app=frontend
```

```
kubectl get pods -l "app=frontend,env=production" # AND
```

```
kubectl get pods -l "version!=v1" # !=
```

ENSEMBLE

```
kubectl get pods -l "app in (frontend, backend)" # IN
```

```
kubectl get pods -l "env notin (dev, test)" # NOT IN
```

```
kubectl get pods -l "team" # EXISTS
```

```
kubectl get pods -l "!canary" # DOES NOT EXIST
```

COMBINAISONS

```
kubectl get pods -l "app=frontend,version!=v1,env in (staging,prod)"
```

Labels et sélecteurs – Syntaxe complète

```
# Dans les YAML (Deployment selector, Service, NetworkPolicy)
selector:
  matchLabels:
    app: frontend                # Sélection simple (AND)
  matchExpressions:
    - key: tier
      operator: In                # In | NotIn | Exists | DoesNotExist
      values: ["web", "api"]
    - key: canary
      operator: DoesNotExist
```


Namespaces – Isolation logique

```
kubectl get namespaces
# default      Namespace par défaut
# kube-system   Composants K8s
# kube-public   Info cluster accessible à tous
# kube-node-lease Heartbeats des nœuds
```

Ce que les namespaces ISOLENT : noms, quotas, NetworkPolicies, rôles
RBAC

Namespaces – Isolation logique

Ce qu'ils N'ISOLENT PAS : réseau (par défaut), nœuds, PV, ClusterRoles

```
# ResourceQuota
apiVersion: v1
kind: ResourceQuota
metadata: {name: dev-quota, namespace: dev}
spec:
  hard:
    pods: "20"
    requests.cpu: "4"
    requests.memory: "8Gi"
    limits.cpu: "8"
    limits.memory: "16Gi"
    persistentvolumeclaims: "10"
```

Namespaces – Isolation logique

```
kubectl create namespace mon-ns  
kubectl config set-context --current --namespace=dev  
kubectl get pods -n dev  
kubectl get pods -A      # Tous les namespaces
```

Stateless vs Stateful – Comparaison

Aspect	Stateless	Stateful
Données	Aucun état local	Données persistantes par instance
Identité	Interchangeables (web-abc12)	Stable (postgres-0 , postgres-1)
Stockage	Pas de PVC dédié	PVC par pod (volumeClaimTemplates)
Scaling up	Parallèle, ordre arbitraire	Ordonné : 0 → 1 → 2
Scaling down	Arbitraire	Ordonné : 2 → 1 → 0
DNS individuel	Non	pod-0.headless.ns.svc.cluster.local
Contrôleur	Deployment	StatefulSet
Exemples	API, frontend, workers	PostgreSQL, Kafka, Redis, Elasticsearch

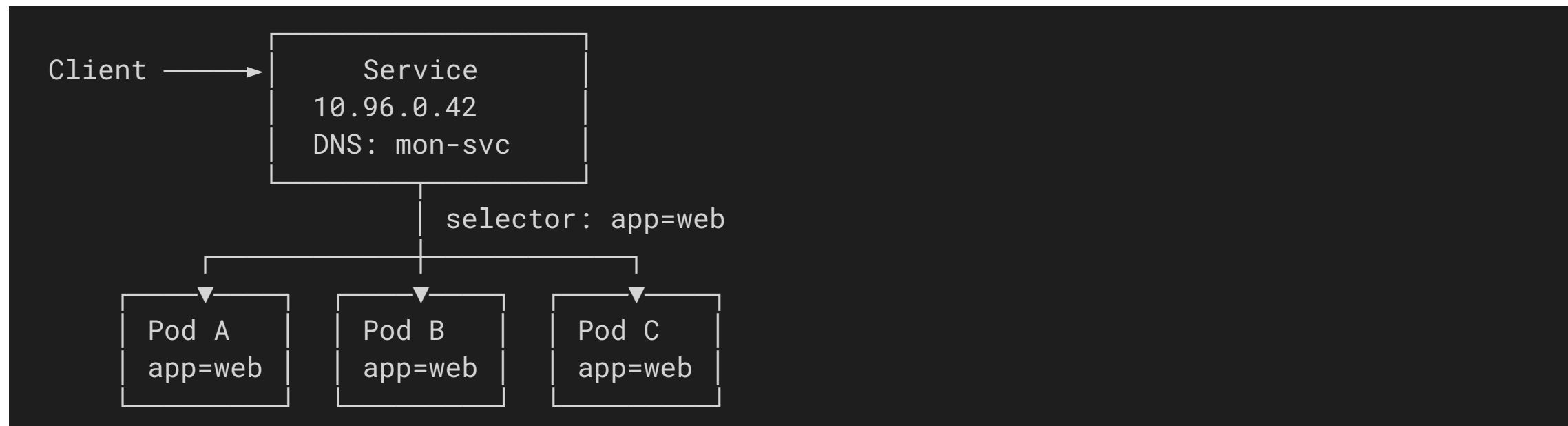
Module 3 — Services

Exposer et découvrir les applications

Le problème des IPs éphémères

Les Pods ont des IPs qui changent à chaque restart/reschedule.

Le **Service** fournit une adresse stable (IP + DNS) devant un ensemble de Pods :



- Sélection des Pods via les **labels**
- **kube-proxy** gère le load balancing (iptables/IPVS)
- Les **Endpoints** sont mis à jour automatiquement quand les Pods changent

Service YAML complet

```
apiVersion: v1
kind: Service
metadata:
  name: mon-service
  namespace: dev
spec:
  type: ClusterIP          # ClusterIP | NodePort | LoadBalancer | ExternalName
  selector:
    app: mon-app          # Tous les pods avec ce label
  ports:
    - name: http
      port: 80             # Port du Service (client)
      targetPort: 8080     # Port du conteneur
      protocol: TCP
    - name: metrics
      port: 9090
      targetPort: 9090
  sessionAffinity: None   # None | ClientIP (sticky sessions)
```

Types de Services

Type	Accès	Port	Cas d'usage
ClusterIP	Interne cluster	IP virtuelle 10.96.x.x	Communication inter-services
NodePort	<NodeIP> :30000-32767	Chaque nœud	Dev, tests
LoadBalancer	IP externe (cloud)	Provisionné par cloud	Production cloud
ExternalName	Alias DNS (CNAME)	Pas de port	Migration, services hors cluster

Types de Services

```
# NodePort
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080          # Accessible : curl localhost:30080

# ExternalName
spec:
  type: ExternalName
  externalName: legacy-db.corp.internal # CNAME, pas de proxy
```

NodePort — Exposition externe aux nœuds

Le **NodePort** ouvre un port fixe sur **chaque nœud** du cluster, permettant l'accès depuis l'extérieur.

Extérieur

`curl http://<NodeIP>:30080`



Nœud 1	:	<NodeIP>:30080	
Nœud 2	:	<NodeIP>:30080	← même port sur tous
Nœud 3	:	<NodeIP>:30080	

kube-proxy DNAT → ClusterIP → Pod



Pod 10.244.x.x:8080

NodePort — Exposition externe aux nœuds

```
apiVersion: v1
kind: Service
metadata:
  name: mon-app-nodeport
spec:
  type: NodePort
  selector:
    app: mon-app
  ports:
    - port: 80          # Port interne (ClusterIP)
      targetPort: 8080  # Port du conteneur
      nodePort: 30080   # Port exposé sur les nœuds (30000-32767)
      protocol: TCP
```

```
# Accéder depuis l'extérieur
curl http://<IP_DU_NOEUD>:30080
```

```
# Sur Kind : le nœud est accessible via localhost
kubectl get nodes -o wide # Récupérer l'IP du nœud
```

Service Discovery et DNS (CoreDNS)

Format complet : <service>.<namespace>.svc.cluster.local

Raccourcis : <service>.<namespace> (tout namespace)

 <service> (même namespace)

Depuis un pod

```
kubectl exec mon-pod -- nslookup mon-service
```

```
kubectl exec mon-pod -- nslookup mon-service.dev.svc.cluster.local
```

```
kubectl exec mon-pod -- curl http://api-service.production # cross-namespace
```

resolv.conf de chaque pod

```
kubectl exec mon-pod -- cat /etc/resolv.conf
```

```
# nameserver 10.96.0.10
```

```
# search dev.svc.cluster.local svc.cluster.local cluster.local
```

```
# ndots:5
```

Headless Service (clusterIP: None)

Retourne les IPs de TOUS les Pods (pas de load balancing) :

```
spec:  
  clusterIP: None  
  selector: {app: postgres}  
  ports: [{port: 5432}]
```

```
# Service normal → IP du ClusterIP
```

```
nslookup mon-service → 10.96.0.42
```

```
# Headless → IPs de chaque Pod
```

```
nslookup postgres-headless → 10.244.1.5, 10.244.2.7, 10.244.3.9
```

Avec StatefulSet — DNS individuel par pod :

```
postgres-0.postgres-headless.default.svc.cluster.local → 10.244.1.5
```

```
postgres-1.postgres-headless.default.svc.cluster.local → 10.244.2.7
```

kube-proxy et Ingress

kube-proxy traduit ClusterIP → IPs des Pods réels (iptables DNAT ou IPVS)

Ingress expose des services HTTP/HTTPS à l'extérieur :

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mon-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
    - hosts: [app.formation.local]
      secretName: tls-secret
  rules:
    - host: app.formation.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
```

Module 4 — Contrôleurs

Gérer le cycle de vie des workloads

Vue d'ensemble des contrôleurs

Contrôleur	Usage	Terminaison	Parent
ReplicaSet	Maintient N réplicas identiques	Arbitraire	Deployment
Deployment	Rolling updates déclaratifs	Arbitraire	—
StatefulSet	Apps avec état, identité stable	Ordonné	—
DaemonSet	Un pod par nœud	Avec le nœud	—
Job	Exécution unique (batch)	À completion	CronJob
CronJob	Exécution planifiée	À completion	—

Deployment — Le contrôleur principal

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: mon-app, namespace: production}
spec:
  replicas: 3
  revisionHistoryLimit: 10
  progressDeadlineSeconds: 600
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # Pods en plus pendant l'update
      maxUnavailable: 0    # 0 = zero-downtime
  selector:
    matchLabels: {app: mon-app}
  template:
    metadata:
      labels: {app: mon-app, version: v1}
    spec:
      containers:
        - name: app
          image: mon-app:v1
          ports: [{containerPort: 8080}]
          resources:
            requests: {cpu: "100m", memory: "128Mi"}
            limits: {cpu: "500m", memory: "256Mi"}
```

Rolling Update visualisé

maxSurge=1, maxUnavailable=0, replicas=3

INITIAL : RS-v1: [A-v1✓] [B-v1✓] [C-v1✓]
RS-v2: (vide)

ÉTAPE 1 : RS-v1: [A-v1✓] [B-v1✓] [C-v1✓] +1 (surge)
RS-v2: [D-v2⌚]

ÉTAPE 2 : RS-v1: [A-v1✓] [B-v1✓] D ready → -1 v1
RS-v2: [D-v2✓]

ÉTAPE 3 : RS-v1: [A-v1✓] [B-v1✓] +1 v2
RS-v2: [D-v2✓] [E-v2⌚]

ÉTAPE 4 : RS-v1: [A-v1✓] E ready → -1 v1
RS-v2: [D-v2✓] [E-v2✓]

ÉTAPE 5-6: RS-v1: [] Scale to 0
RS-v2: [D-v2✓] [E-v2✓] [F-v2✓]

Rolling Update visualisé

```
kubectl set image deployment/mon-app app=mon-app:v2  
kubectl rollout status deployment/mon-app  
kubectl rollout history deployment/mon-app  
kubectl rollout undo deployment/mon-app  
kubectl rollout undo deployment/mon-app --to-revision=3
```

StatefulSet – Applications avec état

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: postgres}
spec:
  serviceName: postgres-headless # OBLIGATOIRE
  replicas: 3
  podManagementPolicy: OrderedReady
  selector:
    matchLabels: {app: postgres}
  template:
    metadata: {labels: {app: postgres}}
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports: [{containerPort: 5432}]
          volumeMounts: [{name: data, mountPath: /var/lib/postgresql/data}]
  volumeClaimTemplates:
    - metadata: {name: data}
      spec:
```

DaemonSet, Job, CronJob

```
# DaemonSet – un pod sur chaque nœud
apiVersion: apps/v1
kind: DaemonSet
metadata: {name: node-exporter, namespace: monitoring}
spec:
  selector: {matchLabels: {app: node-exporter}}
  template:
    metadata: {labels: {app: node-exporter}}
    spec:
      tolerations: [{effect: NoSchedule, operator: Exists}]
      containers:
        - name: exporter
          image: prom/node-exporter
          ports: [{containerPort: 9100, hostPort: 9100}]
```

DaemonSet, Job, CronJob

```
# CronJob – planifié
apiVersion: batch/v1
kind: CronJob
metadata: {name: backup}
spec:
  schedule: "0 2 * * *"           # 2h du matin
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: postgres:16
              command: ["pg_dump", "-h", "postgres", "-U", "admin", "-Fc", "mydb"]
```

Module 5 — Volumes

Persistance des données

Types de volumes

Type	Durée de vie	Partage	Cas d'usage
emptyDir	Durée du Pod	Entre containers du Pod	Cache, tmp, partage sidecar
hostPath	Durée du nœud	Non	DaemonSet, dev
PV/PVC	Indépendant	Selon accessMode	BDD, stockage applicatif
configMap	Auto-refresh ~60s	—	Fichiers de config
secret	Auto-refresh ~60s	—	Certificats, creds
projected	—	—	Combine sources
CSI	Indépendant	Variable	AWS EBS, GCP PD, Ceph

Types de volumes

```
# emptyDir
volumes:
  - name: shared
    emptyDir:
      medium: ""          # "" = disque, "Memory" = tmpfs RAM
      sizeLimit: 500Mi
```

PV et PVC – Provisioning

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: app-data, namespace: dev}
spec:
  accessModes: [ReadWriteOnce]      # RWO | ROX | RWX | RWOP
  resources:
    requests:
      storage: 5Gi
  storageClassName: standard          # Provisioning dynamique
```

Access Mode	Description
RWO	Un seul nœud en R/W
ROX	Multi-nœuds en lecture
RWX	Multi-nœuds en R/W (NFS, CephFS)
RWOP	Un seul Pod en R/W (K8s 1.27+)

Module 6 — Configuration et Secrets

Externaliser la configuration

ConfigMap – Création et injection

```
# Création
kubectl create configmap app-config -n dev \
  --from-literal=DB_HOST=postgres --from-literal=LOG_LEVEL=info

# Injection – 3 méthodes
```

ConfigMap – Création et injection

```
# Méthode 1 : Toutes les clés en env vars
envFrom:
  - configMapRef: {name: app-config}

# Méthode 2 : Clés sélectives
env:
  - name: DATABASE_HOST
    valueFrom:
      configMapKeyRef: {name: app-config, key: DB_HOST}

# Méthode 3 : Volume monté (fichiers, auto-refresh ~60s)
volumeMounts:
  - name: config
    mountPath: /etc/config
volumes:
  - name: config
    configMap:
      name: nginx-conf
      items:
```

Secrets — Types et sécurité

```
kubectl create secret generic db-creds -n dev \  
  --from-literal=username=admin --from-literal=password='K8sForm@tion!'  
kubectl create secret tls app-tls -n dev --cert=tls.crt --key=tls.key  
kubectl create secret docker-registry regcred -n dev \  
  --docker-server=registry.example.com --docker-username=user --docker-password=pass
```

Secrets — Types et sécurité

```
spec:
  imagePullSecrets: [{name: regcred}]
  containers:
    - name: app
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef: {name: db-creds, key: password}
      volumeMounts:
        - name: tls
          mountPath: /etc/tls
          readOnly: true
  volumes:
    - name: tls
      secret:
        secretName: app-tls
        defaultMode: 0400
```

“ Secrets = base64 (pas chiffré). Production : Encryption at Rest + Sealed Secrets ou Vault. ”

Module 7 – Authentification et Identification

Qui peut parler à l'API Server ?

Les deux types de comptes

Type	Objet K8s	Usage	Géré par
User Account	Aucun (externe)	Humains, CI/CD	IdP externe (cert, OIDC, LDAP)
Service Account	ServiceAccount	Pods, controllers	Kubernetes

```
# Service Accounts
kubectl get serviceaccounts -n dev
kubectl describe serviceaccount default -n dev

# Créer un SA dédié
kubectl create serviceaccount mon-app -n dev
kubectl get serviceaccount mon-app -n dev -o yaml
```

Les deux types de comptes

“ Tout pod reçoit automatiquement le SA

`default`

de son namespace.

Le token est monté dans

`/var/run/secrets/kubernetes.io/serviceaccount/token`

”

Certificats X.509 – Authentification utilisateur

L'API Server valide les certificats signés par son CA (

```
/etc/kubernetes/pki/ca.crt
```

).

```
# 1. Générer une clé privée et un CSR
openssl genrsa -out alice.key 2048
openssl req -new -key alice.key \
  -subj "/CN=alice/O=dev-team" \
  -out alice.csr

# 2. Créer un CertificateSigningRequest Kubernetes
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: alice
spec:
  request: $(base64 -w0 alice.csr)
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400
  usages: ["client auth"]
EOF

# 3. Approuver le CSR
kubectl certificate approve alice

# 4. Récupérer le certificat signé
```

Tokens de Service Account

Depuis K8s 1.24, les tokens sont à **durée limitée** (Bound Service Account Tokens).

```
# Token monté automatiquement dans le pod
kubectl exec mon-pod -n dev -- \
  cat /var/run/secrets/kubernetes.io/serviceaccount/token
# Décoder le token JWT
TOKEN=$(kubectl exec mon-pod -n dev -- \
  cat /var/run/secrets/kubernetes.io/serviceaccount/token)
echo $TOKEN | cut -d. -f2 | base64 -d 2>/dev/null | python3 -m json.tool
# Créer un token manuel avec durée limitée (1h)
kubectl create token mon-app -n dev --duration=1h

# Utiliser le token pour appeler l'API depuis un pod
kubectl exec mon-pod -n dev -- sh -c '
  TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
  CA=/var/run/secrets/kubernetes.io/serviceaccount/ca.crt
  curl -s --cacert $CA \
    -H "Authorization: Bearer $TOKEN" \
    https://kubernetes.default.svc/api/v1/namespaces/dev/pods'
```

OIDC – Intégration avec un fournisseur d'identité externe

```
Utilisateur → Login IdP (Keycloak, Azure AD, Google) → JWT id_token
                                                    |
                        kubectl --token=$id_token
                                                    |
                        API Server valide :
                        - Signature JWT
                        - Issuer URL
                        - Audience (aud)
                        - Expiration
```

OIDC – Intégration avec un fournisseur d'identité externe

```
# Configuration API Server (kubeadm / Kind)
# --oidc-issuer-url=https://keycloak.example.com/realms/k8s
# --oidc-client-id=kubernetes
# --oidc-username-claim=email
# --oidc-groups-claim=groups

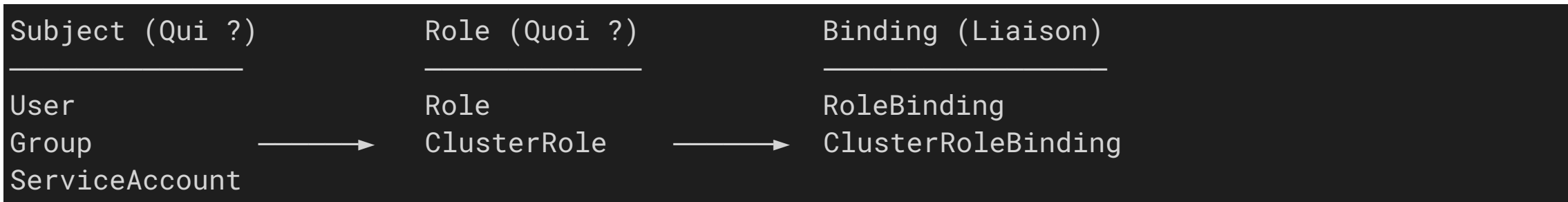
# Côté kubectl (plugin kubelogin)
kubectl oidc-login setup \
  --oidc-issuer-url=https://keycloak.example.com/realms/k8s \
  --oidc-client-id=kubernetes
```

“ **En formation** : simulation avec un certificat X.509 (plus simple à mettre en œuvre sur Kind). ”

Module 8 — Autorisation RBAC

Qui a le droit de faire quoi ?

Modèle RBAC – 4 objets



Objet	Scope	Usage
Role	Namespace	Droits dans un seul namespace
ClusterRole	Cluster entier	Droits globaux ou réutilisables
RoleBinding	Namespace	Lie un Role ou ClusterRole à un Subject (dans un ns)
ClusterRoleBinding	Cluster entier	Lie un ClusterRole à un Subject (partout)

Role et ClusterRole

```
# Role – droits dans le namespace "dev" uniquement
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: dev-reader
  namespace: dev
rules:
  - apiGroups: [""]                # "" = core API (pods, services, configmaps...)
    resources: ["pods", "services", "configmaps"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["apps"]           # apps API (deployments, replicaset...)
    resources: ["deployments", "replicaset"]
    verbs: ["get", "list", "watch"]
```

Role et ClusterRole

```
# ClusterRole – droits sur tout le cluster
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
  - apiGroups: [""]
    resources: ["nodes", "namespaces", "persistentvolumes"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["metrics.k8s.io"]
    resources: ["nodes", "pods"]
    verbs: ["get", "list"]
```

Verbes RBAC

Verbe	HTTP	Action
get	GET	Lire une ressource
list	GET (collection)	Lister des ressources
watch	GET + watch	Stream de changements
create	POST	Créer
update	PUT	Remplacer complètement
patch	PATCH	Modifier partiellement
delete	DELETE	Supprimer
deletecollection	DELETE (collection)	Supprimer plusieurs

Verbes RBAC

```
# Vérifier ses propres droits
kubectl auth can-i get pods -n dev
kubectl auth can-i create deployments -n dev
kubectl auth can-i "*" "*"           # Tout faire ? (admin)

# Vérifier les droits d'un autre utilisateur
kubectl auth can-i get pods -n dev --as=alice
kubectl auth can-i get pods -n dev --as=system:serviceaccount:dev:mon-app
```

RoleBinding et ClusterRoleBinding

```
# RoleBinding – alice peut lire dans dev
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: alice-dev-reader
  namespace: dev
subjects:
  - kind: User
    name: alice
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: dev-reader
  apiGroup: rbac.authorization.k8s.io
```

RoleBinding et ClusterRoleBinding

```
# ClusterRoleBinding – ServiceAccount peut lire les nodes
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: mon-app-node-reader
subjects:
- kind: ServiceAccount
  name: mon-app
  namespace: dev
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

Cas d'usage : ClusterRole lié via RoleBinding

Un **ClusterRole** peut être lié à un namespace via un **RoleBinding** – pratique pour réutiliser des rôles :

```
# Réutiliser le ClusterRole "view" (built-in) dans le namespace dev
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bob-view-dev
  namespace: dev
subjects:
- kind: User
  name: bob
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole      # ClusterRole, mais limité au namespace dev
  name: view             # Role built-in K8s
  apiGroup: rbac.authorization.k8s.io
```

Cas d'usage : ClusterRole lié via RoleBinding

Roles built-in :

ClusterRole	Droits
cluster-admin	Tout faire sur tout
admin	Tout dans un namespace (sauf quotas)
edit	Lire/écrire les workloads
view	Lecture seule (pas les secrets)

```
kubectl get clusterroles | grep -v system:
```


Auditer les RBAC

```
# Qui peut faire quoi sur les pods dans dev ?
kubectl auth can-i --list -n dev

# Tous les RoleBindings d'un namespace
kubectl get rolebindings,clusterrolebindings -n dev -o wide

# Qui a accès à un secret spécifique ?
kubectl get rolebindings -n dev -o json | \
  jq '.items[] | select(.roleRef.name=="dev-reader") | .subjects'

# Décrire un role
kubectl describe role dev-reader -n dev
kubectl describe clusterrole view
```

“ **Bonne pratique** : Principe du moindre privilège — commencer par `view`, ajouter les droits progressivement. ”

Fin du Jour 1

Questions et échanges